# Reached but Not Revealed: How Much Tests See at the Fix Site, and What It Means for Gemma 4 Coding Agents

Author Name
*Affiliation*
City, Country
email

***Abstract*—Coding agents are trained, selected and scored by running tests, so a test that runs a bug without failing is a blind spot in their reward. We trace which tests execute the code a reference fix edits (the fix site) and which fail, on the Gemma 4 Developer Agent dev set, eight libraries, and 104 SWE-rebench tasks from 96 independent repositories, all on CPU. Real bugs are reached but not revealed: in 77 of 78 SWE-rebench bugs, a median of 27 pre-existing tests executed the buggy function and none failed. Of 468 bugs planted at the fix site, the fail-to-pass tests catch 55%, the full benchmark check 76%, and no existing test 18%. With the competition's own Gemma 4 31B checkpoint, candidates taken from runtime traces raise the model's correct first pick to 64%, against 53% from the provided code graph ($p = 0.02$). A failing old test marks a Gemma patch as wrong 96% of the time, but such tests miss 57% of wrong patches. Adding these habits to a competition agent's prompt raised its dev-set resolve rate from 10.4% to 15.6% (106 tasks, two runs per arm; $p = 0.026$). Agents should localize from runtime traces and test the behaviour the issue describes.**

***Index Terms*—software testing, test oracles, coding agents, fault localization, Gemma, SWE-bench**

## I. Introduction

An autonomous coding agent learns whether it succeeded from tests. Benchmarks such as SWE-bench [1] grade a patch by fail-to-pass (F2P) tests written for the issue and pass-to-pass (P2P) tests that must keep working; agents run the repository's tests to check themselves; reinforcement learning turns the same verdicts into reward. All three assume that a test which runs the relevant code would notice if that code were wrong.

Testing theory has long said otherwise. A fault is detected only if it is executed, infects the program state, propagates, and is revealed by an assertion [2], [3]. We ask how large the gap between *reaching* and *revealing* is in the places agents actually edit, and whether it changes what an agent should do. We measure it at the fix site $F^\star$, the functions a task's reference fix edits, and test the consequences with Gemma 4 31B, the model the Gemma 4 Developer Agent competition requires.

Our contributions are:

- a CPU-only, per-test tracing pipeline that measures reach and reveal at $F^\star$, run on 4 competition repositories, 8 libraries and 96 independent SWE-rebench repositories;
- a breakdown of who catches a fault at $F^\star$ (F2P, P2P, other existing tests, nobody), with confidence intervals;
- two experiments with the competition's Gemma 4 checkpoint: which candidates to show the model when localizing, and how well old tests judge its patches;
- concrete guidance for agents, tested as a paired dev-set A/B of a competition agent, plus a 2.7 KB tracer agents can call.

## II. Background and Related Work

**Reach, infection, propagation, reveal.** The PIE model [2] and the RIPR conditions [3] separate executing a fault from detecting it. Pseudo-tested methods, whose bodies can be removed without any test failing [4], [5], and checked coverage [6] show that coverage overstates what tests check. Coverage also correlates only weakly with suite effectiveness once suite size is controlled [7]. We measure the same gap at the fix site of benchmark tasks and on real agent patches.

**Mutants and real faults.** Mutants are a reasonable stand-in for real faults [8], [9], but not a perfect one. We therefore report real bugs and real Gemma patches alongside planted ones.

**Fault localization.** Spectrum-based localization ranks code by how strongly failing tests execute it, for example with Ochiai [10]; PRFL diffuses such scores over a call graph with PageRank [11]. We compare graph sources (runtime traces versus the static graph the competition provides) and test them with an LLM choosing among candidates.

**Benchmark tests are weak.** Patches that pass SWE-bench tests are often wrong [12]; UTBoost [13], SWE-ABS [14] and program-variant probing [15] generate tests that expose them. Agentless filters patches with regression tests [16]. These works generate tests or rerank. We instead measure *which existing test sets* observe the fix site, and how well old tests judge one model's patches.

**Non-normal dynamics.** Non-normal operators can amplify perturbations transiently even when every eigenvalue implies decay [17], [18]. We use this to quantify how far faults could spread through runtime call graphs.

## III. Method

**Datasets.** Table I lists the four datasets. The competition dev set has 129 tasks in four Python repositories (fastapi, rich, requests, httpx); 111 reproduce with their reference

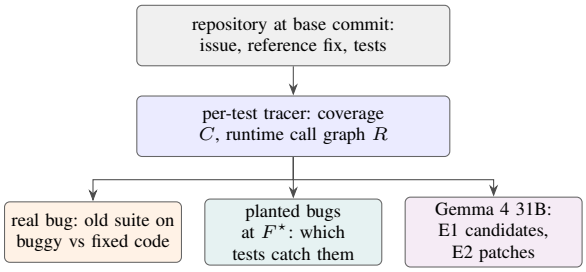



Fig. 1. Pipeline. One trace per task feeds three measurements: the response of the pre-existing suite to the real bug, which test sets catch bugs planted at the fix site $F^\star$, and the context and grading for Gemma 4.

TABLE I
DATASETS

| Dataset | Repos | Tasks used | Planted bugs |
|---|---|---|---|
| Competition dev set | 4 | 101 traced | 1,422 |
| Eight libraries | 8 | full suites | 339 |
| SWE-rebench (independent) | 96 | 104 of 150 | 468 |
| Gemma 4 E1 / E2 (dev set) | 4 | 96–99 / 88 | – |

fix, and 101 can be traced. To test generality we use eight libraries (click, more-itertools, toolz, jinja, itsdangerous, packaging, marshmallow, boltons) at pinned releases, and SWE-rebench [19]. From SWE-rebench we draw 150 candidates (at most two per repository, Python 3.9–3.13, excluding the competition repositories); 104 build and reproduce, from 96 repositories. Everything runs on Kaggle CPUs except Gemma inference.

**Tracing.** A pytest plugin records, for every test, which package functions run, using `sys.monitoring` (Python 3.12+) or `sys.setprofile` with source-derived qualified names (3.9–3.11). This gives a test-by-function coverage matrix $C$ and a runtime call graph $R$ of observed caller–callee edges. Test outcomes are keyed by node id, so selection, tracing and grading use the same identifiers (98.5% of selected tests matched in SWE-rebench mutant runs).

**Real bugs.** We run the pre-existing suite, without the task's test patch, on the buggy and the fixed code. A test *reveals* the bug if it fails on the buggy code and passes on the fixed code. On SWE-rebench every such flip is re-run twice on each side and kept only if it is consistent.

**Planted bugs.** In the fixed code we plant single-site mutants in $F^\star$: comparison flips, Boolean operator swaps, negated conditions, constant changes and `return None`. For each mutant we run the tests that cover it (at most 300, sampled with a fixed seed) plus the F2P and P2P tests. A no-mutation control run flags unstable tests. We record whether the F2P tests, the full harness (F2P and P2P), or any covering test fails.

**Transient amplification.** For a runtime call graph $R$ with spectral radius $\rho$, let $M = R - (\rho + 1)I$. Every mode of $e^{tM}$ decays, yet $A = \max_t \|e^{tM}\|_1$ can exceed 1 when $R$ is non-normal; $A = 1$ for a normal matrix.

**Gemma 4.** We serve the competition's required checkpoint with vLLM 0.19.1 [20] on four L4 GPUs, offline and with

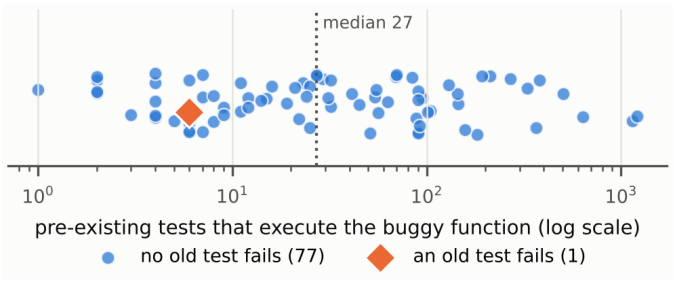



Fig. 2. Real SWE-rebench bugs: for each task where old tests execute the buggy function, how many do. Only one task has an old test that fails on the bug and passes on the fix.

thinking off (model `gemma-4-31b-it-qat-w4a16-ct`). In E1 the model sees the issue and 15 candidate functions in a seeded random order and names up to three to edit (greedy decoding). The candidates come from one of four sources: BM25 ranking of all functions against the issue text, failing-test coverage (Ochiai), coverage plus one hop on $R$, or coverage plus one hop on the competition's provided graph for that commit. In E2 the model gets the issue and the full source of $F^\star$, and returns rewritten functions: one greedy answer and four samples at temperature 0.8. Each distinct patch is graded by the *harness* (all tests in the touched test files that pass with the reference fix) and by *visible* tests: pre-existing tests, not added or edited by the test patch, that execute the edited functions and pass with the reference fix. An agent can run the visible tests itself.

## IV. Reached but Not Revealed

**Real bugs.** In the competition dev set the pre-existing suite executes $F^\star$ in 94 of 101 tasks, a median of 242 tests per task, and reveals the bug in one (93/94 silent). On SWE-rebench (Fig. 2) old tests execute $F^\star$ in 78 of 104 tasks (median 27, IQR 7–90). In 77 of those 78 (99%; 95% CI 96–100%, resampling repositories), no old test fails on the bug and passes on the fix. The exception, in dandi-cli, is a genuine detection by nine tests. Silence is partly expected, since a shipped bug passed its project's CI. What the trace adds is that the suite *ran* the buggy code, so the miss happened at reveal, not at reach.

**Planted bugs at the fix site.** On SWE-rebench, 468 bugs planted in $F^\star$ across 89 tasks split as in Fig. 3. The task's F2P tests catch 55% (95% CI 48–62%), P2P tests add 21 points, other existing tests add 6, and 18% fail no existing test at all. The full benchmark check therefore accepts 24% (18–31%) of fix-site bugs. The F2P share is stable across datasets: 48% on the competition dev set (where any covering test catches 81%) and 51% in a 30-task SWE-rebench pilot. Across all 1,422 competition mutants, 16% of covered mutants make no test fail.

**Spread is large, but does not predict detection.** Runtime call graphs are strongly non-normal. Transient amplification $A$ has a median of 5.1 on the competition repositories (maximum 28 for httpx, whose call graph is acyclic, so every eigenvalue

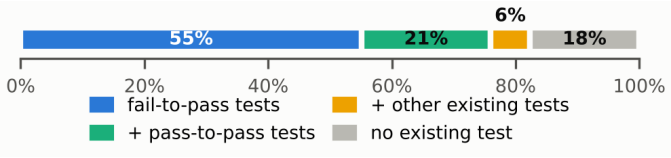



Fig. 3. Who catches a bug planted at the fix site (468 bugs, 89 SWE-rebench tasks; task-weighted). Segments are cumulative test sets.

is zero) and 3.7 on the eight libraries (range 2.0–14.7; 14.7 for itsdangerous, also acyclic). So faults *could* spread far. Which tests notice, however, is governed by observation. The number of tests covering a mutated function predicts how many fail (Spearman ≈0.5–0.6). Graph-only measures (degree, Katz, PageRank, amplification) reach at most 0.35, and none adds information once coverage is known. Detection decayed only weakly with call depth on the competition repositories, and not at all within mutants in the libraries ($p = 0.66$).

**Once a test fails, structure helps trace back.** Ranking functions by failing-test coverage (Ochiai) puts the fix in the top five in 30% of competition tasks, and adding one hop on the runtime graph raises this to 53%. On the same candidates, the runtime graph beats the competition's provided graph: 52% versus 38% top-5 and 28% versus 11% top-1 (90 tasks, $p = 1.4 \times 10^{-6}$). On SWE-rebench, where the fix is usually one hop from what the failing test executes, the extra hop adds nothing (67% to 71%; 32 tasks better, 31 worse).

## V. Gemma 4 31B on the Dev Set

**E1: which candidates to show the model.** Fig. 4 separates retrieval (is the fix among the 15 candidates?) from Gemma's choice (is its first pick the fix?). With runtime-trace candidates, Gemma's first pick is correct in 64% of tasks (95% CI 54–73%). The provided graph gives 53%, failing-test coverage alone 59%, and issue text alone 43%. In paired comparisons on the same tasks, runtime traces beat the provided graph in 13 tasks and lose in 3 ($p = 0.02$, sign test), and beat issue-only retrieval 30 to 10 ($p = 0.002$); both survive a Holm correction. The ordering holds in each of the two repositories that supply most tasks: 57% versus 50% in fastapi (55 tasks) and 72% versus 56% in rich (38). When the fix is among the candidates, Gemma picks it 74–91% of the time, so retrieval is the bottleneck. The provided graph also lowers that conditional accuracy (74% versus 81% for runtime traces): its neighbours look plausible and distract the model.

**E2: how well old tests judge Gemma's patches.** Over 88 tasks and 440 answers, 97% are valid Python. The greedy answer resolves 38% of tasks, sampled answers 41%, and the best of five 45%. Counting F2P tests only gives 39%, so Gemma's failures are almost never "broke something": they are "did not implement the new behaviour". Visible tests exist in 94% of tasks (median 90 per task). Fig. 5 compares the 212 wrong and 178 correct valid patches against them. A visible test fails for 43% of wrong patches (95% CI 31–56%) and 2% of correct ones. A flagged patch is therefore wrong 96% of the time, while a patch that passes every visible test is correct

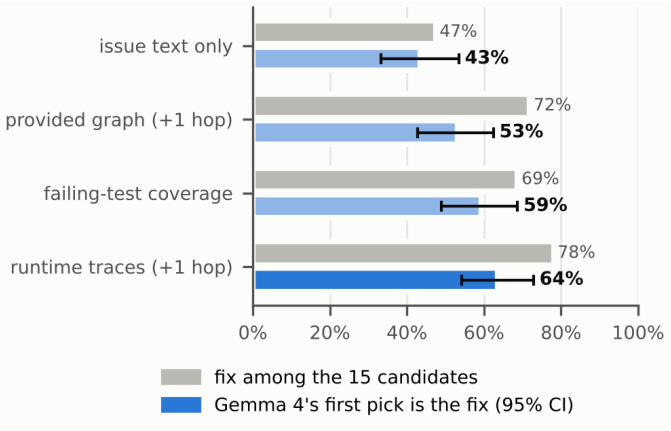



Fig. 4. E1: Gemma 4 31B choosing the function to edit from 15 candidates whose source differs by condition (96–99 dev tasks). Grey: the fix is among the candidates. Blue: Gemma's first pick is the fix.

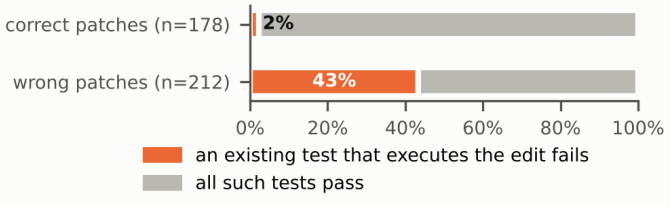



Fig. 5. E2: Gemma 4 31B patches against pre-existing tests that execute the edit (88 dev tasks). Correctness is judged by the harness.

only 59% of the time. Choosing among the five answers by visible tests raises the resolve rate only from 38% to 40%, because answers disagreed in just 9 of 88 tasks.

## VI. Implications for Coding Agents

The three results point the same way. An issue asks for behaviour the project's tests never asserted; otherwise the bug would not have shipped. The old suite therefore reaches the fix site without checking that behaviour, so it cannot reveal the bug and often cannot tell whether a fix worked. What it does check, it checks reliably, which is why a failing old test is such a strong signal. For agents this suggests three habits.

1) *Localize from runtime traces.* Run the issue's reproduction under a tracer and show the model the functions the failing run executed, most suspicious first, not neighbours in a static graph (E1: 64% versus 53%).
2) *Test the issue's behaviour directly.* More than half of wrong patches pass every old test (E2), so re-running the issue's example is the closest proxy for the hidden F2P tests.
3) *Use old tests as a high-precision alarm.* A failing old test that executes the edit is a strong signal to retry (96% precision), but a pass is weak evidence.

**Dev-set A/B.** We added these habits to a competition agent by changing only its system prompt: the agent writes our 2.7 KB tracer, localizes with it, checks the issue's example, and runs one regression check on the tests that execute its edit. We

ran the original prompt (v2, 0.18 on the public leaderboard) and the new one (v3.2) twice each, with different sampling seeds, on the 106 dev tasks that reproduce. Both arms used the same model, a re-implementation of the competition's tools and limits (28 tool calls, 240 s per command, and an 8-minute clock calibrated to single-stream speed), temperature 0.2 and thinking off. Patches were graded by the harness, using only tests that pass twice on the reference fix. Over both runs, v3.2 resolved 15.6% of task runs (33 of 212) against 10.4% for v2 (22 of 212), a gain of 5.2 points (95% CI 1.4–9.4, bootstrap over tasks). v3.2 resolved more often on 14 tasks and v2 on 4 (task-level permutation $p = 0.026$; sign test $p = 0.031$), and each run alone favoured v3.2 (8 to 2 and 7 to 2). The gain comes from precision: when the agent edited, its patch was correct 60% of the time with v3.2 (33 of 55) against 36% with v2 (22 of 61), and wrong patches fell from 35 to 17. The cost is about 1.6 times more generated tokens. Both arms usually exhausted their 28 calls without editing (145 and 157 empty patches out of 212), so call budget, not verification, is now this agent's bottleneck.

For benchmark and reward designers, Fig. 3 is a budget. F2P tests alone leave about half of fix-site faults unobserved, adding P2P halves the remainder, and about a fifth needs new oracles, the gap that generated tests [14], [15] target.

## VII. Limitations

All measurements are on Python with pytest, and the tracer sees only Python-level calls. The Gemma experiments use the four dev-set repositories, greedy or lightly sampled decoding with thinking off, and oracle localization in E2, so absolute resolve rates are not leaderboard rates. The A/B uses our re-implementation of the tool interface rather than the official harness and two samples per arm; six v2 episodes lost their workspace and count as failures. Planted mutants are an imperfect proxy for real faults [8]; we mitigate this with real bugs and real patches, which agree in direction. On SWE-rebench, 33 of 150 candidates failed reproduction, mostly because their F2P tests did not pass even with the fix, which points to environment drift. The SWE-rebench sample is drawn mostly from Python 3.9 tasks.

## VIII. Conclusion

Across 108 repositories, tests reach the code agents edit far more often than they check it. Real bugs pass the suites that run them, a quarter of fix-site faults pass the full benchmark check, and more than half of Gemma 4's wrong patches pass every old test that runs them. The fix is not more graph structure but better observation. Agents should localize from what a failing run executes and test the behaviour the issue describes.

## Data and Code Availability

All notebooks (tracing, SWE-rebench, Gemma prompt pack, inference and grading, figures) and derived data (coverage matrices, call graphs, mutant outcomes, Gemma outputs) are on Kaggle: [link]. Every number in this paper is regenerated by the figures notebook.

## References


[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.

[2] J. M. Voas, "PIE: A dynamic failure-based technique," *IEEE Trans. Softw. Eng.*, vol. 18, no. 8, pp. 717–727, 1992.

[3] P. Ammann and J. Offutt, *Introduction to Software Testing*, 2nd ed. Cambridge Univ. Press, 2016.

[4] R. Niedermayr, E. Juergens, and S. Wagner, "Will my tests tell me if I break this code?" in *Proc. Int. Workshop on Continuous Software Evolution and Delivery*, 2016.

[5] O. L. Vera-Pérez, M. Monperrus, and B. Baudry, "Descartes: A PITest engine to detect pseudo-tested methods," in *Proc. ASE*, 2018.

[6] D. Schuler and A. Zeller, "Assessing oracle quality with checked coverage," in *Proc. ICST*, 2011.

[7] L. Inozemtseva and R. Holmes, "Coverage is not strongly correlated with test suite effectiveness," in *Proc. ICSE*, 2014.

[8] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in *Proc. FSE*, 2014.

[9] M. Papadakis, D. Shin, S. Yoo, and D.-H. Bae, "Are mutation scores correlated with real fault detection?" in *Proc. ICSE*, 2018.

[10] R. Abreu, P. Zoeteweij, and A. J. C. van Gemund, "On the accuracy of spectrum-based fault localization," in *Proc. TAIC PART*, 2007.

[11] M. Zhang, X. Li, L. Zhang, and S. Khurshid, "Boosting spectrum-based fault localization using PageRank," in *Proc. ISSTA*, 2017.

[12] Y. Wang, M. Pradel, and Z. Liu, "Are 'solved issues' in SWE-bench really solved correctly? An empirical study," in *Proc. ICSE*, 2026.

[13] B. Yu *et al.*, "UTBoost: Rigorous evaluation of coding agents on SWE-bench," in *Proc. ACL*, 2025.

[14] B. Yu *et al.*, "SWE-ABS: Adversarial benchmark strengthening exposes inflated success rates on test-based benchmark," arXiv:2603.00520, 2026.

[15] C. Li, Y. Xu, Z. Wang, S. H. Tan, and T.-H. Chen, "Probe to generate: Program variant-guided test augmentation for repository-level repair benchmarks," arXiv:2604.01518, 2026.

[16] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," arXiv:2407.01489, 2024.

[17] L. N. Trefethen and M. Embree, *Spectra and Pseudospectra: The Behavior of Nonnormal Matrices and Operators*. Princeton Univ. Press, 2005.

[18] M. Asllani, R. Lambiotte, and T. Carletti, "Structure and dynamical behavior of non-normal networks," *Science Advances*, vol. 4, no. 12, 2018.

[19] I. Badertdinov *et al.*, "SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents," arXiv:2505.20411, 2025.

[20] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.